

Data Wrangling 🤠

Nested Data Structures

Lesson 14

Out Current Tool

- **Dictionaries** store values by **key** as opposed to by **position**
- We like data we work with to be in the form of a **table** which has many records, each with several labeled fields
- **table** won't fit in just one list or one dictionary

```
# a single college, as a dictionary  
{"Name": "UNC", "City": "Chapel Hill"}
```

Recall: Individuals and Variables

- **Individual** (row): the person/place/thing a record is about
- **Variable** (column): something recorded about every individual

State	Region	Capital City	Population	Land Area	Party	Density
Alabama	South	Montgomery	5024279	50645	Republican	99.21
Alaska	West	Juneau	733391	570641	Republican	1.29
Arizona	West	Phoenix	7151502	113594	Republican	62.96
Arkansas	South	Little Rock	3011524	52035	Republican	57.87
California	West	Sacramento	39538223	155779	Democratic	253.81
Colorado	West	Denver	5773714	103642	Democratic	55.71
Connecticut	Northeast	Hartford	3605944	4842	Democratic	744.72

individual

variable

Row-Oriented: List of Dictionaries

- Each **element** of the list is one **row**
- Order in the list is the row order
- Keys inside each dictionary are the columns

```
colleges = [  
    {"Name": "UNC",      "City": "Chapel Hill"},  
    {"Name": "Duke",     "City": "Durham"},  
    {"Name": "NC State", "City": "Raleigh"},  
]
```

View as Table

- Each **key** becomes a column header; each **value** fills the given column in a row

Name	City
UNC	Chapel Hill
Duke	Durham
NC State	Raleigh

Demo: Accessing Row-Oriented Data

- Index into the **list** first to get a row, then index into the **dictionary** to get a field

```
>>> colleges[0]
{'Name': 'UNC', 'City': 'Chapel Hill'}
>>> colleges[0]["City"]
'Chapel Hill'
>>> colleges[1]["Name"]
'Duke'
```

Demo: Looping over Data

- Loop over the list to visit every row, one at a time

```
for row in colleges:  
    print(f"{row['Name']} is in {row['City']}")
```

Two Ways to Build Tables

- **Row-oriented:** a **list of dictionaries** — one dictionary per individual
- **Column-oriented:** a **dictionary of lists** — one list per variable

Column-Oriented: A Dictionary of Lists

- Each **key** of the dictionary is a **column** name; each **value** is a list of that column's entries, in row order
- Position **i** across every list refers to the same individual

```
colleges = {  
    "Name": ["UNC", "Duke", "NC State"],  
    "City": ["Chapel Hill", "Durham", "Raleigh"],  
}
```

Demo: Accessing Column-Oriented Data

- Index into the **dictionary** first to get a column, then index into the **list** to get one row's value--the order is reversed!

```
>>> colleges["City"]  
['Chapel Hill', 'Durham', 'Raleigh']  
>>> colleges["City"][0]  
'Chapel Hill'  
>>> colleges["Name"][1]  
'Duke'
```

Concept Check

Given the column-oriented `colleges` dictionary, what does each expression evaluate to?

```
colleges = {  
    "Name": ["UNC", "Duke", "NC State"],  
    "City": ["Chapel Hill", "Durham", "Raleigh"],  
}
```

```
colleges["City"]  
colleges["City"][2]  
len(colleges["Name"])
```

Which one do we use?

	Row-oriented (list of dicts)	Column-oriented (dict of lists)
Best for	working with one record at a time	pulling values from one variable at once
Natural loop	<pre>for row in colleges:</pre>	<pre>for value in colleges["City"]:</pre>
Adding a record	<pre>.append(new_row_dict)</pre>	update every list, one per column

Demo: In Favor of Row-Oriented

Question: Which college is in Raleigh?

```
raleigh_colleges = []
for row in colleges:
    if row["City"] == "Raleigh":
        raleigh_colleges.append(row["Name"])

print(raleigh_colleges)
```

- We need to look at every field of a row together (`City` *and* `Name`)
— row-oriented keeps them attached

Demo: In Favor of Column-Oriented

Question: What are the colleges' cities, in alphabetical order?

```
cities = colleges["City"] # dict-oriented version  
print(sorted(cities))
```

- We only ever need **one column** — column-oriented hands it to us directly, no digging through rows

Concept Check

Fill in the blank so that `columns` ends up as

```
{ 'Name': [ 'Duke', 'NC State' ], 'City': [ 'Durham', 'Raleigh' ] }:
```

```
rows = [  
    { "Name": "Duke", "City": "Durham" },  
    { "Name": "NC State", "City": "Raleigh" },  
]
```

```
columns = { "Name": [], "City": [] }
```

```
for row in rows:  
    columns["Name"].____(row["Name"])  
    columns["City"].____(row["City"])
```

Demo: Converting Row-Oriented → Column-Oriented

```
rows = [  
    {"Name": "UNC", "City": "Chapel Hill"},  
    {"Name": "Duke", "City": "Durham"},  
]  
  
columns = {"Name": [], "City": []}  
  
for row in rows:  
    columns["Name"].append(row["Name"])  
    columns["City"].append(row["City"])
```


Another Combination

- Sometimes you want row-oriented data, but looked up by a **meaningful key** instead of position
- A **dictionary of dictionaries**, keyed by something unique like a college's name:

```
by_college = {  
    "UNC": {"City": "Chapel Hill"},  
    "Duke": {"City": "Durham"},  
}
```

```
>>> by_college["Duke"]["City"]  
'Durham'
```

Nesting Recap

- Data structures can **nest** inside each other
- The type at each "layer" tells you how to access it:
 - Inside a `list` → index by **position**: `data[0]`
 - Inside a `dict` → index by **key**: `data["Name"]`
- Read a nested expression **left to right**, one layer at a time

```
colleges[0]["City"]      # list, then dict
colleges["City"][0]      # dict, then list
by_college["UNC"]["City"] # dict, then dict
```